



Mixture of Chapters

Scaling Learnt Memory in Transformers

Tasmay Pankaj Tibrewal (22MF10041)

Indian Institute of Technology Kharagpur

Co-authors: Pritish Saha, Ankit Meda (IIT KGP); Kunal Singh, Pradeep Moturi (Fractal AI)

Accepted at ICLR Workshop NFAM 2026

B.Tech Project – II Viva, 2026



Motivation

Transformers have no explicit memory

Transformers are powerful sequence models, but knowledge is stored **implicitly** in dense parameters — hard to inspect, edit, or scale independently.

Existing workarounds operate *outside* the model:

- KV-cache retention / compression for long context.
- Retrieval-Augmented Generation (RAG) over external corpora.
- Tool-use, recurrence over hidden states.

Research Question

Can we add a **learned, persistent memory module** *inside* the transformer — and **scale** it sparsely without blowing up compute?



Three Goals

What MoC is designed to achieve

Explicit memory A learned, addressable bank that complements parametric capacity — inspired by PKM and Memory Layers at Scale.

Scalable sparse access Dense cross-attention over a huge bank is expensive. Borrow the Mixture-of-Experts routing principle and apply it to memory.

Retention under continued training An anchored memory bank should resist catastrophic forgetting across training-phase transitions (e.g. pretrain → instruction FT).

Headline contribution

Mixture of Chapters (MoC): a transformer block with a *262,208-token* learned memory bank, partitioned into *4,097 chapters*, with sparse top-*k* routing per sequence.



Method (1/2)

Memory layer: a learned bank read by cross-attention

We augment a decoder-only transformer with a memory bank $M \in \mathbb{R}^{N_m \times d}$ of latent tokens, trained end-to-end.

The token stream produces queries; memory provides keys and values:

$$\begin{aligned} Q &= \text{Norm}(H) W_Q, \quad K = M W_K, \quad V = M W_V \\ \text{MemRead}(H, M) &= \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V \\ H &\leftarrow H + \text{MemRead}(H, M) \end{aligned}$$

- Memory tokens are random-initialized, learned by gradient descent.
- Knowledge lives in a **compact latent space**, not retrieved text.
- Drop-in inside a standard transformer block.



Method (2/2)

Mixture of Chapters: scaling via sparse routing

Bottleneck: dense attention over a very large bank is prohibitive ($\mathcal{O}(L N_m)$ per memory layer).

Idea: partition M into C chapters of T tokens ($N_m = C T$), and route each input sequence to a top- k subset.

$$r = \text{Pool}(H) \in \mathbb{R}^d,$$

$$p = \text{softmax}(W_r r + b_r)$$

$$S = \text{TopK}(p, k),$$

$$M_S = \text{Concat}(\{M_c : c \in S\})$$

The memory layer then attends only to M_S : $H \leftarrow H + \text{MemRead}(H, M_S)$.

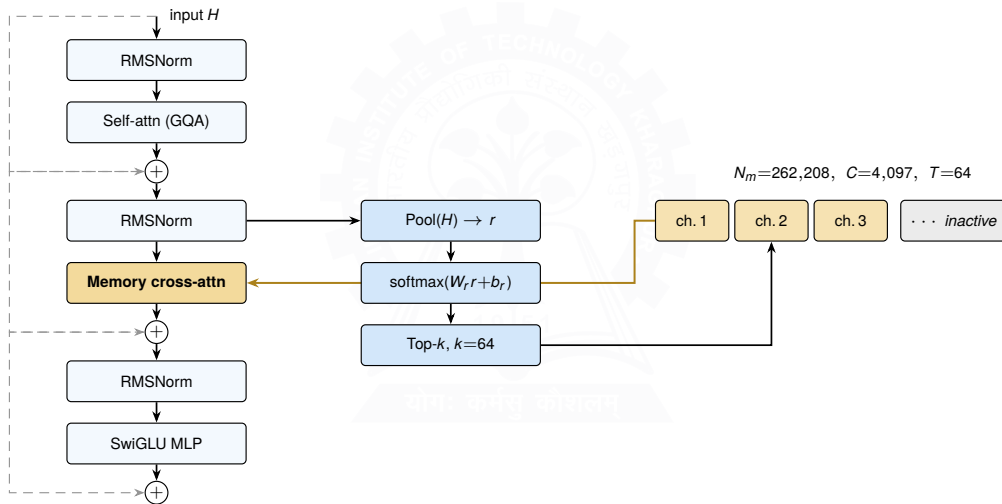
Cost reduction

$\mathcal{O}(L N_m) \rightarrow \mathcal{O}(L k T)$ per memory layer. With $N_m=262,208$, $k=64$, $T=64$: only $\sim 1.6\%$ of the bank touched per sequence.



Architecture

A single transformer block with chapter-routed memory





Experimental Setup

Iso-FLOP comparison, heavy post-training

Models compared:

- **MoC** – our memory-augmented transformer.
- **Vanilla (iso-FLOP)** – dense baseline compute-matched to MoC.
- **Vanilla (backbone-only)** – the same backbone with memory components removed.

Training protocol

- Pretrain: **9.6B tokens** (9,600 steps, context 1024).
- Instruction fine-tune (IFT): **2 epochs** on 230M tokens (3.2k steps, context 2048).
- Heavy post-training budget — deliberately induces forgetting in the dense baseline.

Evaluation: MMLU, ARC-Challenge, BoolQ, OpenBookQA.



Result 1: Pretraining

MoC achieves the lowest validation loss

Model	Validation loss ↓
Vanilla (backbone-only)	2.92
Vanilla (iso-FLOP)	2.86
Mixture of Chapters (MoC)	2.79

- MoC *beats* the iso-FLOP dense baseline at the same compute.
- Curves continue to separate late in training $\Rightarrow$ additional headroom with longer runs.
- Suggests **learned memory is a complementary axis of scaling** — not just a re-allocation of existing capacity.



Result 2: Knowledge Retention under IFT

Heavy post-training crushes the dense baseline; MoC holds

Benchmark	Vanilla (iso-FLOP)			MoC		
	Pretrain	Δ	IFT	Pretrain	Δ	IFT
MMLU	26.90	-0.99	25.91	27.87	-0.35	27.52
ARC-C	31.77	-6.69	25.08	31.44	-2.68	28.76
BoolQ	57.13	-6.24	50.89	61.87	+0.24	62.11
OBQA	36.20	-2.00	34.20	37.40	-2.00	35.40

- Vanilla collapses on knowledge-heavy tasks: BoolQ *near-random*, ARC-C drops 6.7 pp.
- MoC: **BoolQ unchanged**, ARC-C drops <half as much.
- Consistent with memory bank *anchoring* factual knowledge.



Bonus: Freezing the Bank

The pretrained memory can be reused as-is during IFT

We compare three IFT regimes for the memory bank: **Implication**

1. Frozen (no updates).
2. Trained at *very small* learning rate.
3. Trained at *full* backbone learning rate.

- Pretrained bank can be *reused* during post-training without further updates.
- The backbone shifts toward instruction alignment; the bank *anchors* the facts.
- Suggests a **separation of roles** between adaptable backbone and stable memory.

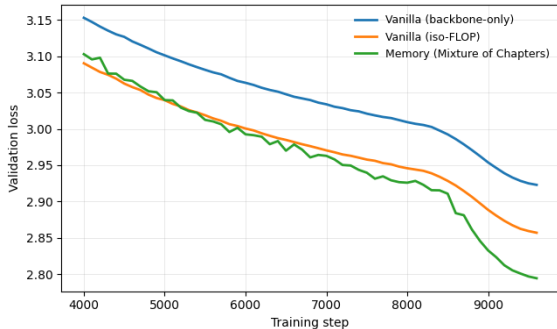
Finding

IFT validation curves **overlap**, post-IFT benchmarks **within noise**.

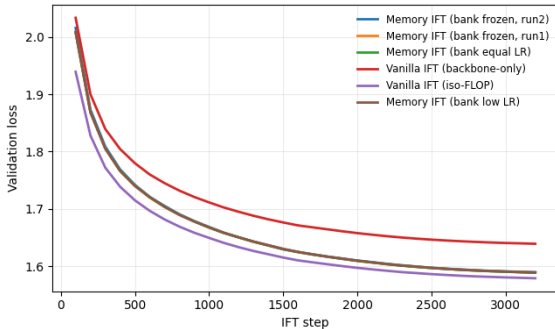


Loss Curves

Visual evidence: pretraining separation & IFT-bank overlap



Pretraining eval loss (9,600 steps). MoC keeps descending; iso-FLOP and backbone-only flatten.



IFT eval loss — bank-freeze and LR variants overlap; freezing yields identical curves.



Beyond the Paper

Improved config + extended training: gap keeps widening

Setup	Tokens	Loss gap vs iso-FLOP
Workshop paper	9.6B	0.063
Improved (LR + bank activation)	9.6B	0.135 (2.14 $\times$)
Extended (paper config, 1.5 $\times$ tokens)	14.4B	0.090 (1.43 $\times$)

- Curves *intersect* around step 5,250; post-intersection budget $\sim 2.1 \times \Rightarrow \sim 1.43 \times$ gain — **close to linear scaling**.
- In *both* runs the gap is **still widening**, no saturation.

Implication

Learned memory is a **genuine complementary scaling axis** — not a bonus that plateaus early. Even stronger gains expected at larger token budgets.



Conclusion

Learned memory as a new axis of scaling

- **Explicit, trainable memory bank** accessed by cross-attention, scaled to **262K** tokens via chapter routing.
- **Lower validation loss** than compute-matched dense baselines — gap doubles with tuned config, widens with longer training.
- **Robust to forgetting** under heavy IFT — BoolQ and ARC-C stable where vanilla collapses.
- Bank can be **frozen** during post-training without loss $\Rightarrow$ clean separation between knowledge retention and task adaptation.

Take-away

A learned, attention-accessed associative memory bank is a **complementary axis of scaling** — distinct from PKM / Memory Layers at Scale (sparse FFN substitutes) and from RAG-style external retrieval.



Future Work (1/2)

Benchmarking and scaling — targeting full paper venues

1. **Head-to-head benchmarks** vs Memory Layers at Scale, UltraMem, and Product-Key Memory at *matched* compute — including retention evaluations that none of these papers currently report.
2. **Scaling laws for learned memory** — sweep bank capacity $\times$ training tokens to formalize the trend already observed in the post-paper runs.
3. **Token-level routing** (currently sequence-level) with efficient kernels for finer-grained, per-token memory access.

Closest comparison

Memory Layers at Scale (Meta FAIR) is the clearest apples-to-apples comparison and our top priority for the full paper.



Future Work (2/2)

Interpretability and broader evaluation

4. Interpretability of chapter specialization

- Router selection patterns across topics / domains.
- Causal interventions (zeroing / swapping chapters) to localize what is stored where.

5. Broader evaluation suites

- Factual recall benchmarks.
- Long-context generalization.
- Multi-stage post-training retention beyond IFT.

Beyond BTP

This BTP delivers the architecture, the workshop paper, and early scaling evidence. The full paper targets **EMNLP 2026 / ICLR 2027** with matched-compute baselines and scaling laws.



Acknowledgements & Questions

Sincere thanks to:

- Mentor: Kunal Kingkar and Supervisor: Prof. Pawan Goyal, for guidance through this BTP.
- Co-authors Prithish Saha, Ankit Meda (IIT KGP); Kunal Singh, Pradeep Moturi (Fractal AI).
- ICLR Workshop NFAM 2026 reviewers for the accept and feedback.
- Compute and tooling providers used during the project (Neysa, Modal).
- Fractal AI, Modal for the compute sponsor.

Code: github.com/Tasmay-Tibrewal/Memory

Questions & Discussion

Thank you.

Compute & support:



neysa